

Lekcja

Temat: Wstęp do algorytmów.

Algorytm to uporządkowany, jednoznaczny zestaw kroków prowadzących do rozwiązania konkretnego problemu lub wykonania zadania.

Przykład (z życia codziennego)

Algorytm robienia herbaty:

1. Zagotuj wodę
2. Włóż torebkę herbaty do kubka
3. Zalej wrzątkiem
4. Odczekaj 3–5 minut
5. Wyjmij torebkę

Zalety algorytmów

1. **Precyzyjność** – jasno określone kroki minimalizują błędy.
2. **Powtarzalność** – zawsze dają ten sam wynik dla tych samych danych wejściowych.
3. **Automatyzacja** – umożliwiają działanie programów komputerowych.
4. **Oszczędność czasu** – dobrze zaprojektowany algorytm przyspiesza pracę.
5. **Rozwiązywanie złożonych problemów** – dzielą duże problemy na mniejsze kroki.

Wady algorytmów

1. **Brak elastyczności** – działają tylko według zaprogramowanych reguł.
2. **Mogą być nieefektywne** – źle zaprojektowany algorytm działa wolno.
3. **Złożoność** – trudne algorytmy są skomplikowane do zrozumienia i utrzymania.
4. **Wymagają dokładnych danych wejściowych** – błędne dane = błędny wynik.
5. **Koszt projektowania** – stworzenie dobrego algorytmu może być czasochłonne.

Lekcja

Temat: Podstawowe algorytmy

Dodać implementacje w C++

1. Sortowanie bąbelkowe (Bubble Sort)

Wyjaśnienie: To prosty algorytm **sortowania**, który wielokrotnie przechodzi przez listę, porównując sąsiednie elementy i zamieniając je miejscami, jeśli są w złej kolejności. Proces **powtarza się**, aż lista będzie posortowana. Jest nieefektywny dla dużych zbiorów danych (złożoność $O(n^2)$), ale łatwy do zrozumienia. Nazwa pochodzi od "bąbelków" – większe elementy "wypływają" na koniec listy.

Przykład kodu Python:

```
def bubble_sort(arr):
    n = len(arr) #
    for i in range(n): # Pętla zewnętrzna: dla każdej iteracji
        swapped = False # Flaga do optymalizacji (czy była zamiana)
        for j in range(0, n - i - 1): # Pętla wewnętrzna: porównaj sąsiednie elementy
            if arr[j] > arr[j + 1]: # Jeśli bieżący > następny, zamień

                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

Jeśli tak, wykonujesz zamianę: większy element „przesuwa się w prawo”, a mniejszy „przesuwa się w lewo”.

```
lista = [64, 34, 25, 12, 22, 11, 90]
posortowana = bubble_sort(lista)
print(posortowana)
```

Przykład kodu C++:

```
#include <iostream>
#include <vector>

void bubble_sort(std::vector<int>& arr) {
    int n = arr.size(); //
    for (int i = 0; i < n; ++i) { // Pętla zewnętrzna: dla każdej iteracji
```

```

bool swapped = false; // Flaga do optymalizacji (czy była zamiana)
for (int j = 0; j < n - i - 1; ++j) { // Pętla wewnętrzna: porównaj sąsiednie
    elementy
        if (arr[j] > arr[j + 1]) { // Jeśli bieżący > następny, zamień

            /*
                Jeśli tak, wykonujesz zamianę: większy element „przesuwa się w prawo”, a
                mniejszy „przesuwa się w lewo”.
            */
            std::swap(arr[j], arr[j + 1]);
            swapped = true;
        }
    }
    if (!swapped) {
        break;
    }
}

int main() {
    std::vector<int> lista = {64, 34, 25, 12, 22, 11, 90};
    bubble_sort(lista);
    for (int num : lista) {
        std::cout << num << " ";
    }
    std::cout << std::endl;
    return 0;
}

```

2. Wyszukiwanie binarne (Binary Search)

Wyjaśnienie: Algorytm **wyszukuje element w posortowanej liście, dzieląc ją na pół za każdym razem. Porównuje szukany element z środkowym elementem listy i eliminuje połowę, w której elementu nie ma.** Jest bardzo efektywny (złożoność $O(\log n)$), ale wymaga, aby lista była posortowana. To jak szukanie słowa w słowniku – otwierasz na środku i sprawdzasz, czy iść w lewo czy w prawo.

Przykład kodu:

```

def binary_search(arr, target, low=0, high=None):
    if high is None:
        high = len(arr) - 1 # Inicjalizacja high na końcu listy
    if low > high:

```

```

        return -1 # Nie znaleziono

mid = (low + high) // 2 # Środkowy indeks
if arr[mid] == target:
    return mid # Znaleziono
elif arr[mid] < target:
    return binary_search(arr, target, mid + 1, high) # Prawa połowa
else:
    return binary_search(arr, target, low, mid - 1) # Lewa połowa

posortowana_lista = [11, 12, 22, 25, 34, 64, 90]
indeks = binary_search(posortowana_lista, 25)
print(indeks) # Wyjście: 3 (indeks elementu 25)

```

Przykład kodu w C++

```

#include <iostream>
#include <vector>

int binary_search(const std::vector<int>& arr, int target, int low = 0, int high = -1) {
    if (high == -1) {
        high = arr.size() - 1; // Inicjalizacja high na końcu listy
    }
    if (low > high) {
        return -1; // Nie znaleziono
    }

    int mid = low + (high - low) / 2; // Środkowy indeks (unikanie przepełnienia)
    if (arr[mid] == target) {
        return mid; // Znaleziono
    } else if (arr[mid] < target) {
        return binary_search(arr, target, mid + 1, high); // Prawa połowa
    } else {
        return binary_search(arr, target, low, mid - 1); // Lewa połowa
    }
}

int main() {
    std::vector<int> posortowana_lista = {11, 12, 22, 25, 34, 64, 90};
    int indeks = binary_search(posortowana_lista, 25);
    std::cout << indeks << std::endl; // Wyjście: 3 (indeks elementu 25)
    return 0;
}

```

Albo wersja iteracyjna (bez rekurencji, lepsza dla dużych list, bo unika przepełnienia stosu):

```

def binary_search_iterative(arr, target):
    low = 0

```

```

high = len(arr) - 1
while low <= high:
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
return -1 # Nie znaleziono

# Przykład:
posortowana_lista = [11, 12, 22, 25, 34, 64, 90]
indeks = binary_search_iterative(posortowana_lista, 64)
print(indeks) # Wyjście: 5

```

3. Obliczanie silni (rekurencyjnie)

Silnia (oznaczana jako $n!$) to iloczyn wszystkich dodatnich liczb całkowitych od 1 do n . Na przykład, $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$. Algorytm rekurencyjny oblicza silnię poprzez wywoływanie samej siebie z mniejszym argumentem, aż dojdzie do przypadku bazowego.

```

def silnia(n):
    if n <= 1: # Przypadek bazowy
        return 1
    else:
        return n * silnia(n - 1) # Krok rekurencyjny

# Przykład użycia:
wynik = silnia(5)
print(wynik) # Wyjście: 120

# Inne przykłady:
print(silnia(0)) # 1
print(silnia(1)) # 1
print(silnia(6)) # 720

```

Jak to działa?

- Dla `silnia(5)`: Wywołuje `5 * silnia(4)`
- `silnia(4)`: Wywołuje `4 * silnia(3)`
- `silnia(3)`: Wywołuje `3 * silnia(2)`
- `silnia(2)`: Wywołuje `2 * silnia(1)`
- `silnia(1)`: Zwraca `1` (przypadek bazowy)
- Teraz "wspinamy się" z powrotem: $2 * 1 = 2$, $3 * 2 = 6$, $4 * 6 = 24$, $5 * 24 = 120$.

Symulacja wykonania dla $n=3$:

- `silnia(3) → 3 * silnia(2)`

- $\text{silnia}(2) \rightarrow 2 * \text{silnia}(1)$
- $\text{silnia}(1) \rightarrow 1$
- Wracając: $\text{silnia}(2) = 2 * 1 = 2$, $\text{silnia}(3) = 3 * 2 = 6$.

Przykład kodu w C++

```
#include <iostream>

int silnia(int n) {
    if (n <= 1) { // Przypadek bazowy
        return 1;
    } else {
        return n * silnia(n - 1); // Krok rekurencyjny
    }
}

// Przykład użycia:
int main() {
    int wynik = silnia(5);
    std::cout << wynik << std::endl; // Wyjście: 120
    return 0;
}
```

4. Wyszukiwanie liniowe (Linear Search)

Wyszukiwanie liniowe to najprostszy algorytm wyszukiwania elementu w liście (tablicy). Polega na sprawdzaniu każdego elementu po kolei, od początku do końca, aż znajdzie się poszukiwany element lub dojdzie do końca listy. Nie wymaga, aby lista była posortowana, co jest jego zaletą w porównaniu do wyszukiwania binarnego.

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1 #

lista = [64, 34, 25, 12, 22, 11, 90]
indeks = linear_search(lista, 25)
print(indeks)

owoce = ["banan", "jabłko", "gruszka", "pomarańcza"]
indeks = linear_search(owoce, "jabłko")
print(indeks)

indeks = linear_search(owoce, "wiśnia")
print(indeks)
```

Przykład kodu w C++

```
#include <iostream>
#include <vector>
#include <string>

template <typename T>
int linear_search(const std::vector<T>& arr, const T& target) {
    for (size_t i = 0; i < arr.size(); ++i) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1; //
}

int main() {
    std::vector<int> lista = {64, 34, 25, 12, 22, 11, 90};
    int indeks = linear_search(lista, 25);
    std::cout << indeks << std::endl;

    std::vector<std::string> owoce = {"banan", "jabłko", "gruszka", "pomarańcza"};
    indeks = linear_search(owoce, "jabłko");
    std::cout << indeks << std::endl;

    indeks = linear_search(owoce, "wiśnia");
    std::cout << indeks << std::endl;
    return 0;
}
```

Lekcja

Temat: Notacja asymptotyczna: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$

Notacja asymptotyczna (Big O Notation) – to sposób, w jaki opisujemy **jak bardzo algorytm spowalnia się** wraz ze wzrostem ilości danych (oznaczanej jako **n**).

Big O mówi: „w najgorszym przypadku algorytm wykona maksymalnie tyle operacji”.

Nie interesuje nas dokładna liczba operacji (np. $5n + 3$), tylko **zachowanie dla bardzo dużych n** (np. 1 milion rekordów).

Ignorujemy:

- stałe mnożniki ($5n \rightarrow n$),
- niższe potęgi ($n^2 + 3n + 10 \rightarrow n^2$).

1. **$O(1)$ – stała złożoność (stały czas)**

Czas wykonania jest **zawsze taki sam**, niezależnie od ilości danych. Nawet jeśli danych będzie milion razy więcej, algorytm wykona tę samą liczbę operacji. Zawsze tyle samo operacji.

Przykład:

- odczyt elementu z tablicy: **arr[5]**
- dodanie dwóch liczb
- sprawdzenie czy liczba jest parzysta ($x \% 2 == 0$)

Nawet jeśli masz 10 miliardów elementów – operacja trwa tyle samo.

Przykład z życia: Wzięcie konkretnej książki z półki, jeśli znasz jej dokładny adres (np. "trzecia półka, piąta od lewej"). Nie musisz przeglądać wszystkich książek, po prostu sięgasz we wskazane miejsce.

2. **$O(\log n)$ – logarytmiczna**

Czas wykonania rośnie bardzo powoli wraz ze wzrostem danych. Za każdym razem, gdy wykonujemy krok, odrzucamy połowę danych (lub znaczną ich część). Podstawa logarytmu (najczęściej 2) nie ma w notacji znaczenia.

Przykład:

- **Wyszukiwanie binarne** (Binary Search) w posortowanej tablicy
- wyszukiwanie w posortowanej bazie danych

Dla $n = 1\,000\,000$ wystarczy tylko 20 porównań!

Przykład z życia: Szukanie słowa w książce telefonicznej. Nie czytasz od deski do deski. Otwierasz książkę w połowie, widzisz, że szukane słowo jest wcześniej, odrzucasz tylną połowę i znów otwierasz pozostałą część w połowie. Po kilku krokach znajdujesz właściwą stronę.

3. **$O(n)$ – liniowa**

Czas wykonania rośnie **proporcjonalnie** do rozmiaru danych. Jeśli danych będzie 2 razy więcej, algorytm będzie działał 2 razy dłużej. Przechodzisz przez dane **raz**.

Przykład:

- **Wyszukiwanie liniowe** (Linear Search)
 - Przejście pętlą for po wszystkich elementach tablicy (np. aby znaleźć maksymalną wartość),
 - suma elementów tablicy
 - wydrukowanie wszystkich rekordów
- 10× więcej danych = 10× więcej czasu.

Przykład z życia: Szukanie konkretnego tytułu na nieposortowanym stosie książek. Musisz sprawdzić książkę po książce od góry do dołu. Jeśli książek jest 100, sprawdzisz max. 100, jeśli 200, to max. 200.

4. **$O(n \log n)$ – liniowo-logarytmiczna**

Trochę wolniejsze od liniowego, ale znacznie szybsze od kwadratowego. To "złoty środek" dla wydajnych algorytmów sortowania. Najlepsza możliwa złożoność dla **sortowania**

Przykład:

- **Quick Sort** - jest algorytmem typu "dziel i zwyciężaj" i osiąga średnią złożoność **$O(n \log n)$** dzięki temu, że za każdym razem dzieli problem na mniejsze części i robi to sprawnie.
- **Merge Sort** (sortowanie przez scalanie),

Zasada działania Merge Sort

Algorytm działa następująco:

1. **Dziel (Divide):** Podziel tablicę na dwie równe (lub prawie równe) połowy.
2. **Zwyciężaj (Conquer):** Rekurencyjnie posortuj każdą połowę.
3. **Połącz (Merge):** Scal dwie posortowane połowy w jedną posortowaną tablicę.

- sortowanie w Pythonie (`sorted()`) i w C++ (`std::sort`)
To złoty standard dla dużych zbiorów danych.

Przykład z życia: Sortowanie dużej talii kart przez łączenie (jak w sortowaniu przez scalanie). Dzielisz talię na mniejsze, sortujesz je, a potem sprawnie łączysz.

5. **$O(n^2)$ – kwadratowa**

Czas wykonania rośnie **wraz z kwadratem** ilości danych. Jest to algorytm bardzo wolny dla dużych n . Dla 10 elementów to 100 operacji, ale dla 1000 elementów to już 1,000,000 operacji.

Przykład:

- Zagnieżdżone pętle (dwie pętle for jedna w drugiej) wykonujące się do n .
- **Bubble Sort** (bąbelkowe) - jest klasycznym przykładem algorytmu o złożoności **$O(n^2)$** , ponieważ używa **dwóch zagnieżdżonych pętli**, które w najgorszym wypadku wykonują liczbę operacji proporcjonalną do kwadratu liczby elementów.
- **Insertion Sort** (przez wstawianie)
- **Selection Sort** (sortowanie przez wybór) - ma złożoność **$O(n^2)$** z tego samego powodu co Bubble Sort — używa dwóch zagnieżdżonych pętli. Jednak robi to w nieco inny sposób, co sprawia, że jest w pewnym sensie "bardziej uczciwym" $O(n^2)$, bo (w przeciwieństwie do Bubble Sort) nie ma wersji "optymalnej", która by go przyspieszała.
Dla 10 000 elementów to już **100 milionów** operacji – zaczyna być wolno.

Przykład z życia: Porównywanie każdego z każdym. Np. chcesz sprawdzić, czy w grupie osób są dwie o takim samym imieniu. Bierzesz pierwszą osobę i porównujesz ze wszystkimi pozostałymi, potem drugą osobę ze wszystkimi (z pominięciem pierwszej) itd.

6. **$O(2^n)$ – wykładnicza**

Czas wykonania **podwaja się** z każdym dodanym elementem. Nawet niewielki wzrost danych powoduje gigantyczny wzrost czasu. Dla dużych n jest to praktycznie niemożliwe do obliczenia. Najgorsza możliwa praktyczna złożoność. Czas rośnie **eksplozjowo**.

Przykład:

- **Fibonacci bez memoizacji** (często nazywany "naiwnym Fibonaccim") to algorytm obliczania ciągu Fibonacciego przy użyciu czystej rekurencji, bez zapamiętywania wyników pośrednich.
Jeśli wywołasz `fibonacci(5)`, funkcja wywoła `fibonacci(4)` i `fibonacci(3)`.
Te z kolei wywołają kolejne. Szybko okazuje się, że **ciągle liczymy to samo wiele razy**.

```

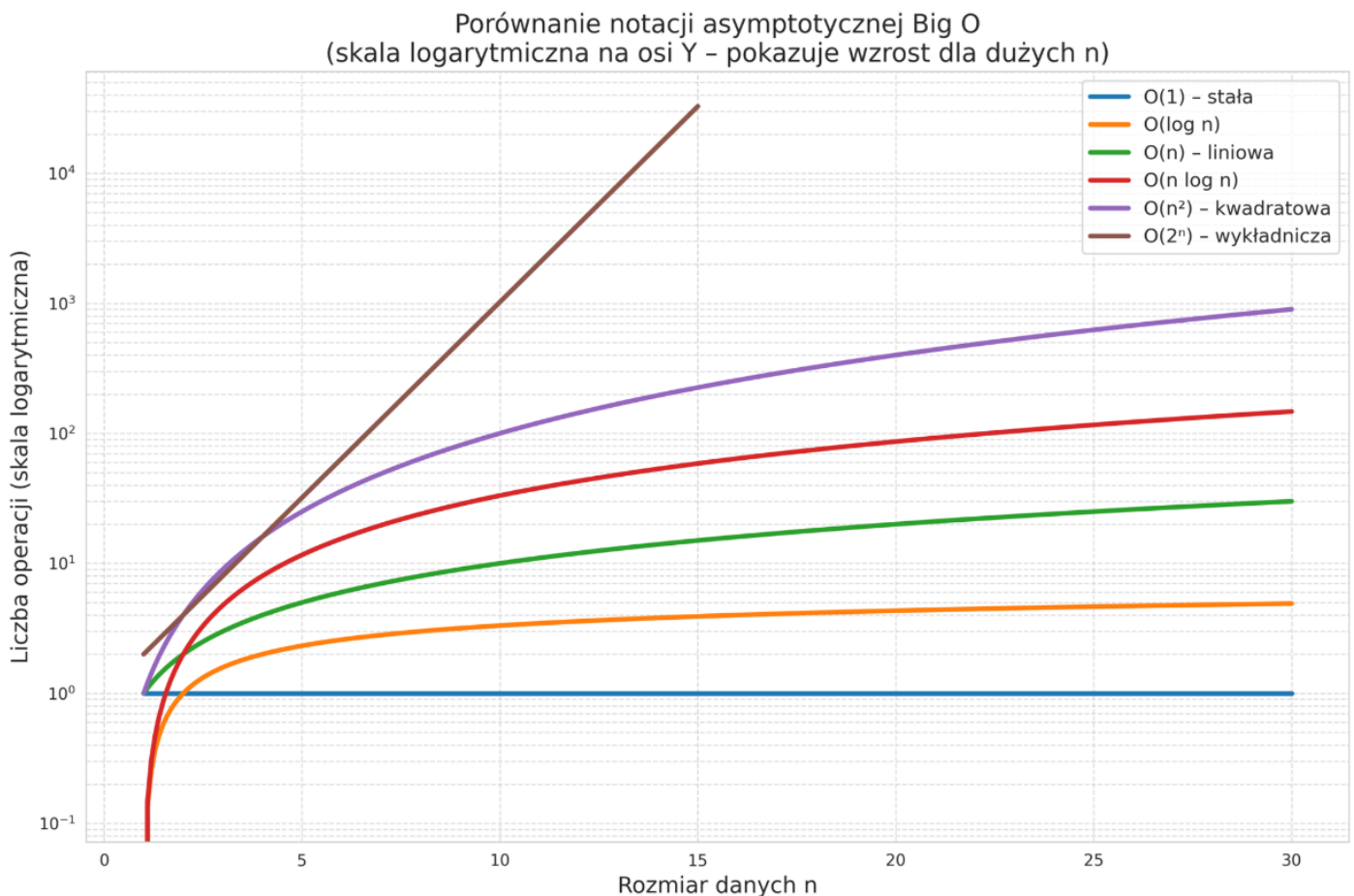
fibonacci(5) // <- chcemy obliczyć 5
├── fibonacci(4)
│   ├── fibonacci(3)
│   │   ├── fibonacci(2)
│   │   │   ├── fibonacci(1) -> 1
│   │   │   └── fibonacci(0) -> 0
│   │   └── fibonacci(1) -> 1
│   └── fibonacci(2) // <- TO JUŻ LICZYLIŚMY!
│       ├── fibonacci(1) -> 1
│       └── fibonacci(0) -> 0
└── fibonacci(3) // <- TO TEŻ JUŻ LICZYLIŚMY!
    ├── fibonacci(2)
    │   ├── fibonacci(1) -> 1
    │   └── fibonacci(0) -> 0
    └── fibonacci(1) -> 1

```

- problem komiwojażera rozwiązywany metodą "brute force" (sprawdzenie wszystkich możliwych tras), wieże Hanoi.
- generowanie wszystkich podzbiorów zbioru
- brute-force rozwiązanie problemu komiwojażera
Dla $n = 30$ to już ponad **1 miliard** operacji. Dla $n = 100$ – niemożliwe do obliczenia nawet na superkomputerze.

Przykład z życia: Przewidywanie wszystkich możliwych ruchów w partii szachów. Z każdym kolejnym ruchem liczba możliwych scenariuszy eksploduje.

Oto porównanie wszystkich klas na jednym wykresie (oś Y w skali logarytmicznej, żeby było widać różnicę):



- Linie, które rosną powoli ($O(1)$, $O(\log n)$, $O(n)$) – **świetne algorytmy**.
- Linie, które idą mocno w górę ($O(n^2)$ i zwłaszcza $O(2^n)$) – **algorytmy tylko dla małych danych**.